



THE UNIVERSITY of EDINBURGH
Centre for Data, Culture & Society



Foundations of Machine Learning

Attendance



March 6th (2026)

Session 1

Dr. Somya Iqbal



www.cdcs.ed.ac.uk

Course plan

2 hour sessions mix of teaching and hands-on practice

Session 1: Lead **Somya Iqbal**

Introduction to ML & data

Codebook 1a

Classification & regression

Codebook 1b

Summary and conclusions

Q & A

Session 2: Lead tutor **Joy Lan**

Classification models cont..

Codebook 1a

Regression & practical considerations

Codebook 1b

Summary and conclusions

Q & A



Session 1

10:00-10:20: Introduction to Machine Learning

10:20 -10:45: **Codebook 1a**

Break

5-minutes: *Run through of solutions*

10:55 -11:15: Classification & regression models

11:15 -11:45: **Codebook 1b**

5-minutes: *Run through of solutions*

11:45 – 11:55: Summary and conclusions

Last 5 minutes – wrap up & Q & A



Introduction to Machine Learning & Data Exploration

- Machine learning - > '**learning**'
- Training with algorithms
- Making predictions or classifications (inferences) from learned patterns without explicit programming
- Mathematical functions and computer programming as mediums



Pre-processing data

- Standardising entries, wrangling and sub-setting based on goal and question
- Missing values, skewness, redundant variables (noise)
- Choosing models based on underlying assumptions
- Garbage in -> garbage out (e.g. unrepresentative, size, variable metrics do not relate to desired outputs)



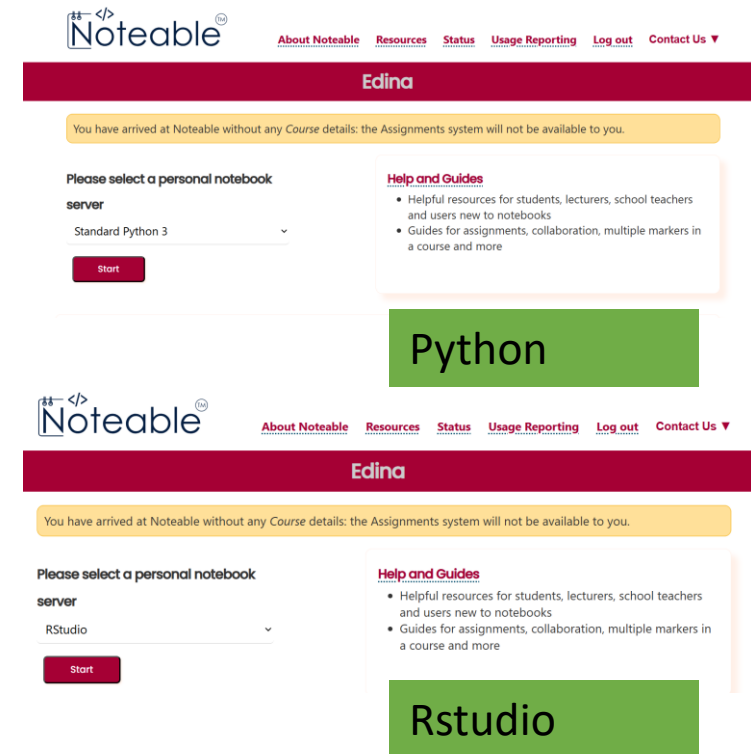
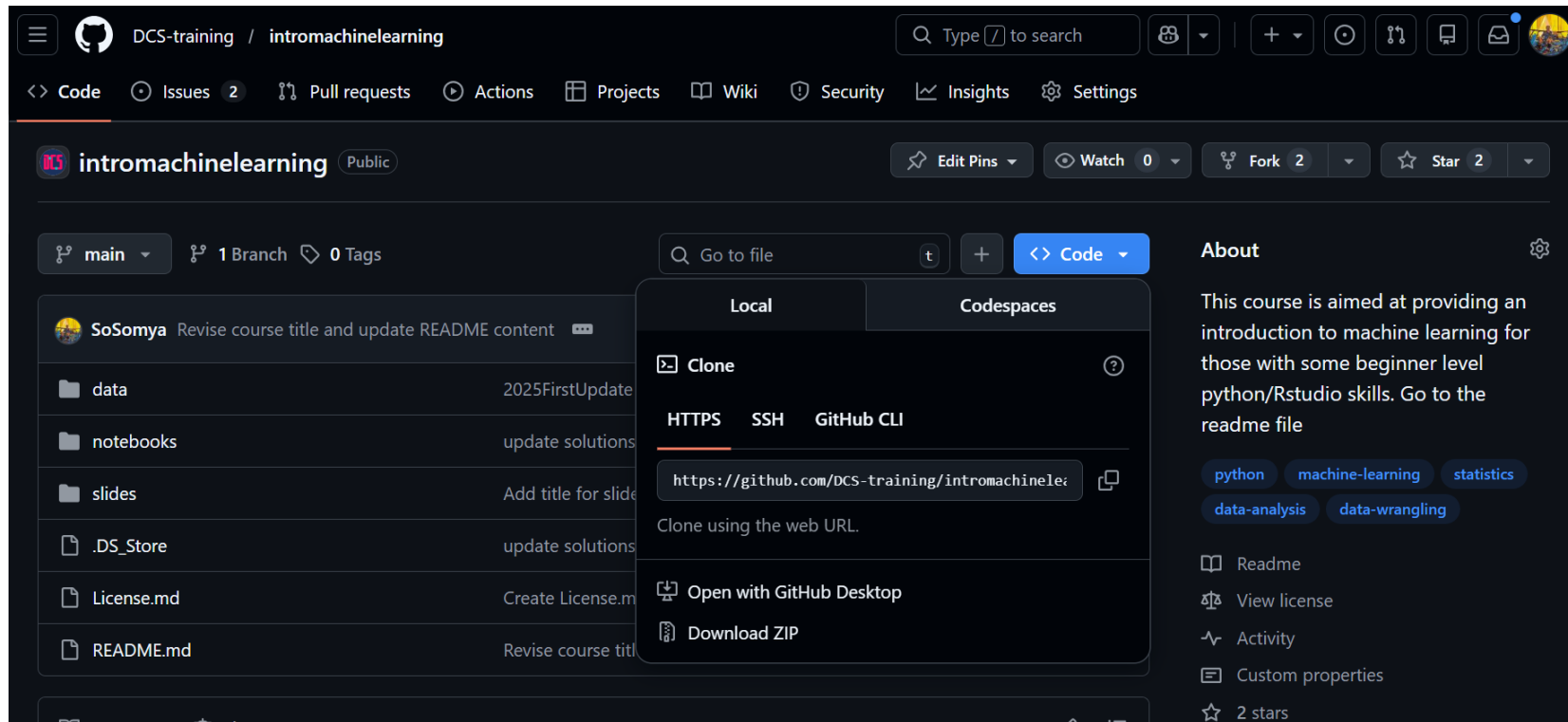
Inspecting data & quality assurance

- Visually inspect distributions
- Summary statistics & descriptors
- Assessing quality & relevance of variables



Setting up repository clone for exercises

Makes a clone – you can add notes & change items



Markdown (text formatting language)

What is this code block doing?

1. Base URL:
<https://collectionapi.metmuseum.org/public/collection/v1/> is the ****root**** for all API access.
2. Endpoint /search:
This is used to find object IDs based on keywords. The URL becomes:
<https://collectionapi.metmuseum.org/public/collection/v1/search>
 - Note the addition of ****search**** in the URL
3. Query parameters is ****q**** in the code block. Additionally 'hasImages' was included to ensure that items with images are included in the parameters.
 - > Note: after some exploration, it it became apparent that removing this element draws on less results '96', this is interesting and may be related to image data generally having fuller records or fuller metadata. A consideration when making the search, to be precise where there are specific aims.
4. We check the status code
5. Inspecting what we have and printing out some details to check before we proceed.

All the sentences in green with a ******* are annotations we have added to explain the code block and what is being executed.

markdown

Define the search term - lets say we want to search items relat

What is this code block doing?

1. Base URL:
<https://collectionapi.metmuseum.org/public/collection/v1/> is the **root** for all API access.
2. Endpoint /search: This is used to find object IDs based on keywords. The URL becomes:
<https://collectionapi.metmuseum.org/public/collection/v1/search>
 - Note the addition of **search** in the URL
3. Query parameters is **'q'** in the code block. Additionally 'hasImages' was included to ensure that items with images are included in the parameters.
 - Note: after some exploration, it it became apparent that removing this element draws on less results '96', this is interesting and may be related to image data generally having fuller records or fuller metadata. A consideration when making the search, to be precise where there are specific aims.
4. We check the status code
5. Inspecting what we have and printing out some details to check before we proceed.

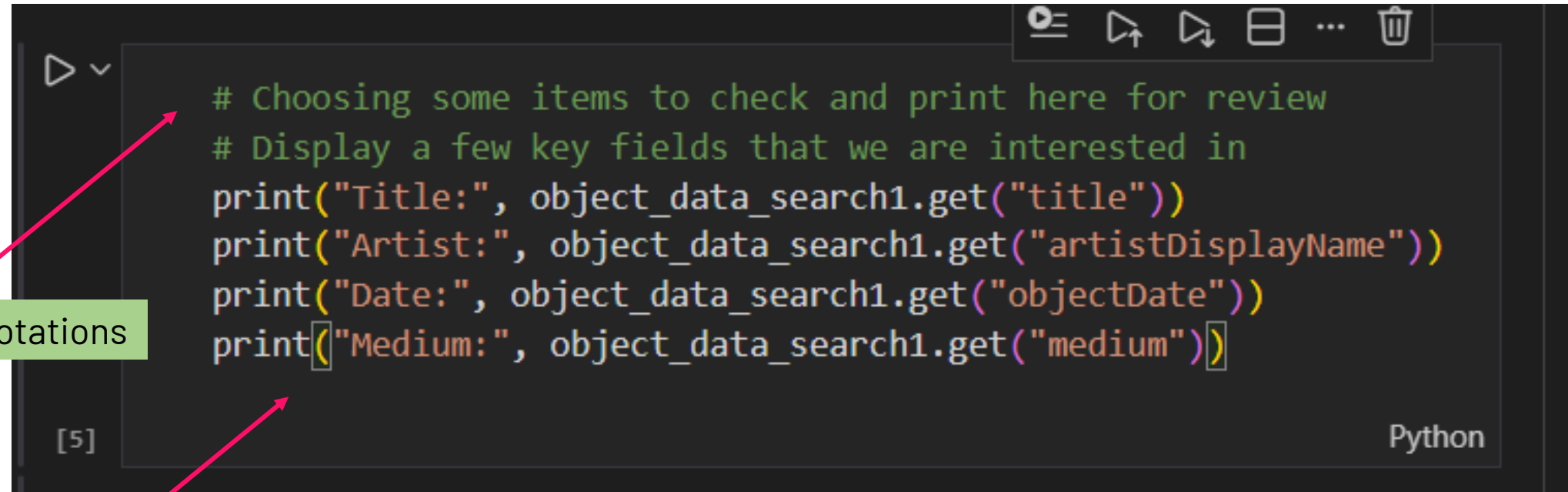
All the sentences in green with a # are annotations we have added to explain the code block and what is being executed.

How we format text

Output preview



Code cells



The image shows a Jupyter Notebook code cell with a dark background. The code is written in Python and includes comments. The cell has a toolbar at the top with icons for running, saving, and other actions. The code is as follows:

```
# Choosing some items to check and print here for review
# Display a few key fields that we are interested in
print("Title:", object_data_search1.get("title"))
print("Artist:", object_data_search1.get("artistDisplayName"))
print("Date:", object_data_search1.get("objectDate"))
print("Medium:", object_data_search1.get("medium"))
```

The cell is labeled [5] in the bottom left corner and Python in the bottom right corner.

Helpful annotations

Executable lines of code

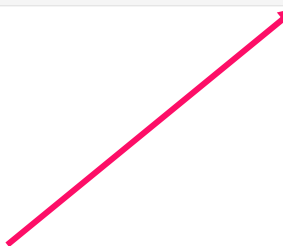


SESSION CODEBOOK (1a)

- Work sequentially block by block
- Note where annotation **#green** gives a hint for you to complete entries
- Wait for the cell to run and review the outputs

```
# EXERCISE: Fill missing values for 'Age' with the median and 'Embarked' with the mode.
# Hint: Use .fillna() method for both columns.

titanic_data['Age'].fillna(____, inplace=True) # Replace ____ with appropriate value
titanic_data['Embarked'].fillna(____, inplace=True) # Replace ____ with appropriate value
```



Complete the fields



SESSION (1a) - SOLUTIONS



ACTIVITY FEEDBACK

Why do missing values
matter here?

```
# EXERCISE: Check for missing values in each column.  
# Hint: Use .isnull().sum() to get the count of missing values.  
titanic_train.isnull().sum()
```

```
# EXERCISE: Plot survival by passenger class using sns.countplot with hue='Survived'.  
plt.figure(figsize=(6, 4))  
sns.countplot(x='Pclass', hue='Survived', data=titanic_train)  
plt.title('Survival by Passenger Class')  
plt.xlabel('Passenger Class')  
plt.ylabel('Count')  
plt.legend(title='Survived')  
plt.show()
```

```
# EXERCISE: Plot survival by sex to see how it affects survival rates.  
plt.figure(figsize=(6, 4))  
sns.countplot(x='Sex', hue='Survived', data=titanic_train)  
plt.title('Survival by Sex')  
plt.xlabel('Sex')  
plt.ylabel('Count')  
plt.legend(title='Survived')  
plt.show()
```

Variables & plots (counts)



Classification basics & Logistic Regression

Problem framing: Student performance (pass/fail) → Features (e.g. study hours)

Training

Learning with supervision

Prediction

Input = features
labels = outputs

The goal is to predict the label in new instances

Formally →

A learning algorithm that implements a set of functions over the instance space, whose outputs are in the label space. The application of the learning algorithm to a data set of labelled instances, $\mathcal{D} = \{(\mathbf{x}^1, c^1), \dots, (\mathbf{x}^N, c^N)\}$, will provide a supervised classification model

Looks for patterns in labelled data

Finds boundaries between classes



Common classifiers:

Logistic Regression

Decision Trees

k-NN

SVM

Neural Networks (deep learning)

Slide source: Chris Oldnall

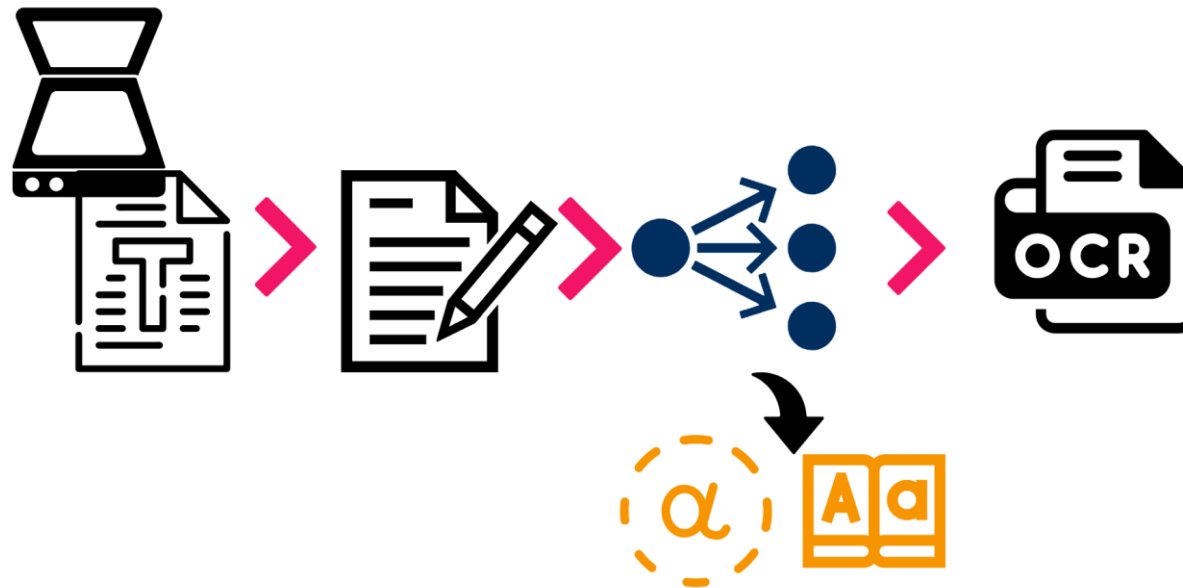
Talk

Labelled Training data

Machine Learning Model

Prediction

9 7 0 2 5 2 / 1 6 8 8
6 7 8 8 8 7 0 0 \ 1 / 1
3 0 / 1 2 3 4 5 6 7 8
9 0 \ 1 2 3 4 5 6 7 8
0 / 1 2 5 6 0 9 9 0 6 / 1
3 4 5 6 7 8 9 0 / 1 2 2
6 7 8 9 0 / 1 2 3 4 5 5
/ 1 / 1 / 1 / 1 2 3 4 5 5 6 6
4 6 4 5 6 7 8 9 9 9
9 9 8 9 9 9 1 2 4 5
6 7 8 9 4 5 6 3 3 2
3 3 3 3 8 4 4 9 6 0
9 0 0 6 5 8 9 5 6 8
8 / 1 8 9 9 4 6 7 8 9
7 2 2 5 7 8 3 6 4 1



OCR as an example – simplified to ***is it a 't' or an 'l'?*** – teaching the program letter recognition – known labels from the hand transcribed elements

Machine learning models look at features like these to learn patterns

Features in data

Features = height, cross lines
Label = letter

Learned pattern

Tall + 1 cross $\rightarrow$ t
Tall + 0 cross $\rightarrow$ l

Prediction

t

Image of letter



Extract features

(height, crossings,
pixels, curves....)



Machine learning model



Predicted label

(t, l, a, etc.)

Training Data

- Representative sample of data
- Desired output from a program
- Used to develop ML program

Caution: Algorithmic bias & context suitability



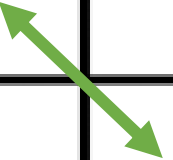
Evaluation → classifier

ACTUALLY → ground truth

PREDICTED → from Model

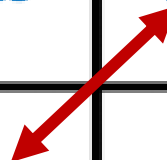
Confusion Matrix

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)



Confusion Matrix

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)



Evaluation -> classifier

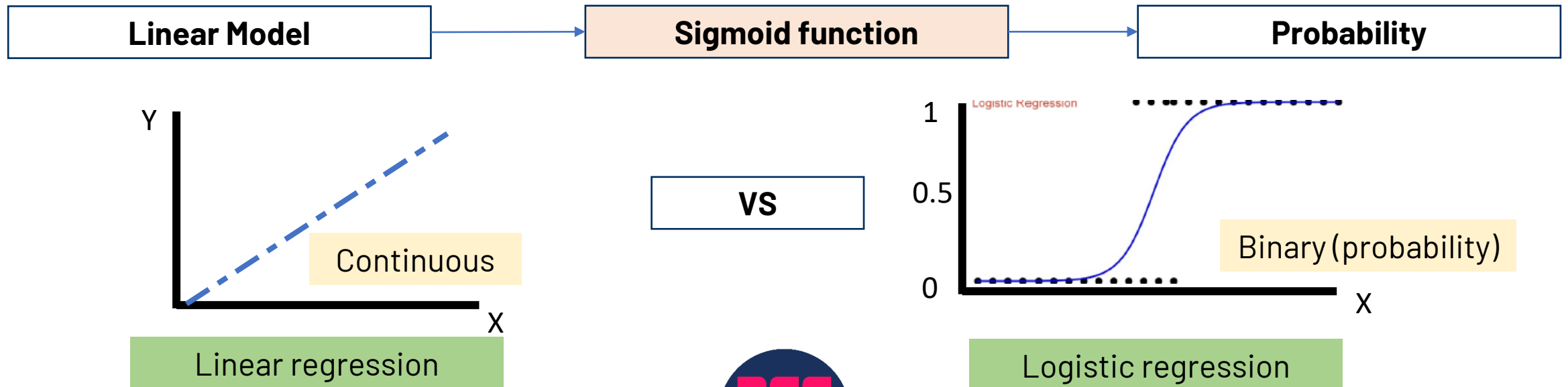
Methods need metrics for evaluating performance, given the data and task that was assigned.

1. **Accuracy** - The **proportion** of **total predictions** the model got right $(TP + TN) / total$
2. **Precision** - Of the cases the model **predicted as positive**, how many were actually positive? If the model predicts email is spam, precision tells us how often that prediction is correct. $TP / (TP + FP)$
3. **Recall** - The proportion of **actual positives** that the model correctly identified. Of all the actual positive cases, how many did the model successfully identify? $TP / (TP + FN)$



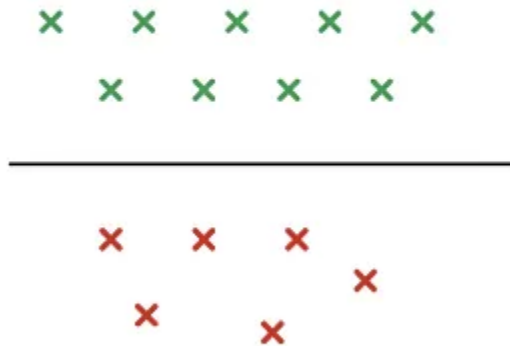
Logistic Regression

- Binary classification
- We predict a **probability of belonging to a class**
- Logistic regression predicts a value between **0 and 1** -> **assign class**

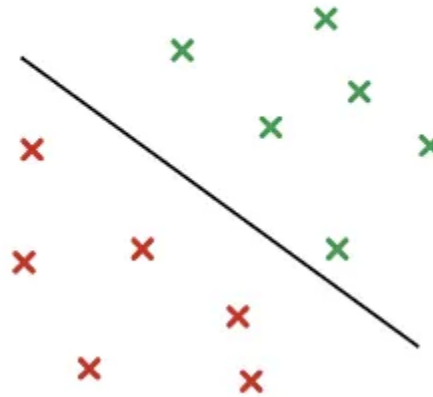


Logistic Regression

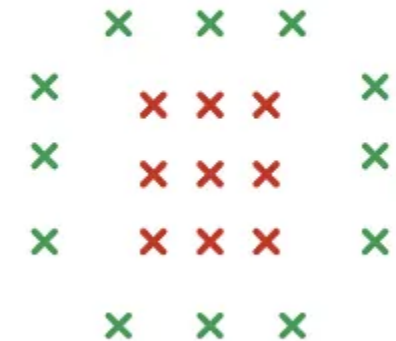
✓ Case-I



✓ Case-II



✗ Case-III



SESSION CODEBOOK (1b)

- Work sequentially block by block
- Note where annotation **#green** gives a hint for you to complete entries
- Wait for the cell to run and review the outputs

```
# EXERCISE: Fill missing values for 'Age' with the median and 'Embarked' with the mode.
# Hint: Use .fillna() method for both columns.

titanic_data['Age'].fillna(____, inplace=True) # Replace ____ with appropriate value
titanic_data['Embarked'].fillna(____, inplace=True) # Replace ____ with appropriate value
```

Complete the fields



SESSION (1b) - SOLUTIONS



ACTIVITY FEEDBACK

```
: # Fill missing values for 'Age' with median
titanic_data['Age'].fillna(titanic_data['Age'].median(), inplace=True)

# Fill missing values for 'Embarked' with mode
titanic_data['Embarked'].fillna(titanic_data['Embarked'].mode()[0], inplace=True)
```

Median & mode (impute)

```
: # EXERCISE: Split the dataset into training (80%) and testing (20%) sets.
# Use train_test_split function with test_size=0.2 and random_state=42.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.8, random_state=42)
```

Training and test data
setup

```
# EXERCISE: Train the Logistic Regression model on the training data.
# Hint: Use .fit() method with X_train and y_train.

log_reg.fit(X_train, y_train)
```

Fitting model/training

```
LogisticRegression(max_iter=1000)
```

```
# EXERCISE: Predict the survival on the test data using the trained model.
# Hint: Use .predict() method with X_test.

y_pred = log_reg.predict(X_test)
```

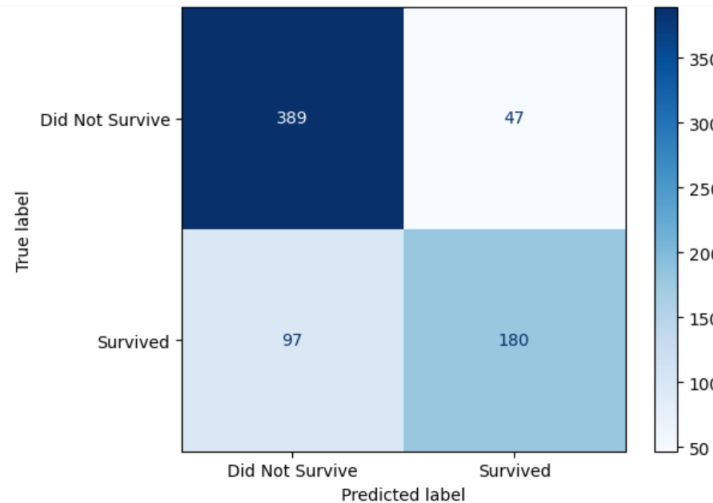
Predict



ACTIVITY FEEDBACK

```
# EXERCISE: Plot the confusion matrix.  
# Hint: Use ConfusionMatrixDisplay with confusion_matrix(y_test, y_pred).  
  
cm = confusion_matrix(y_test, y_pred)  
ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Did Not Survive', 'Survived']).plot(cmap='Blues')  
plt.show()
```

A confusion matrix shows how many predictions were correct and incorrect for each class.



Further ML methodologies

Unsupervised ML: Clustering, dimension reduction, anomaly detection

Reinforcement learning

Neural networks & Deep learning



Next week with Joy

Classification – KNN & Decision Trees

Pitfalls & considerations



Feedback form



<https://forms.office.com/r/YYNrquvNr8>

